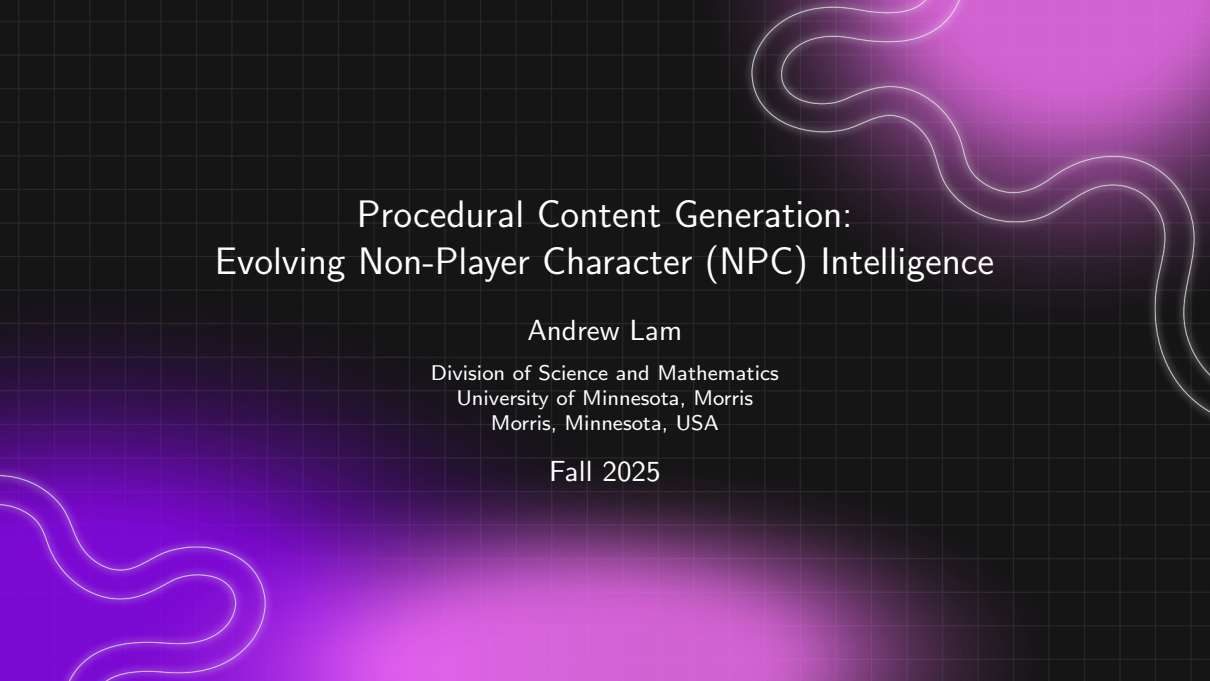




Procedural Content Generation: Evolving Non-Player Character (NPC) Intelligence

Andrew Lam

Division of Science and Mathematics
University of Minnesota, Morris
Morris, Minnesota, USA

Fall 2025

The Big Picture

- A non-player character (NPC) is any character controlled by the game (not the human player), serving as opponents, allies, or a character in the game.
- Making Non Player Characters could be **time-consuming, costly**
- Hand-scripted behaviors for NPC also require lots of **effort, hard to scale** for big game with various types of NPC.
- A tool automate parts of NPC behavior design → EvolvingBehavior tool (Genetic Programming to evolve Behavior Trees)

Outline

- Background Concepts
- EvolvingBehavior (Co-Creative Evolution)
- How the tool works
- Experiments
- Results
- Conclusion

Outline

Background

Evolving Behavior

How the tool works

Experiments

Results

Conclusion

Background Concepts

- Procedural Content Generation (PCG)
- Behavior Trees in Game AI
- Evolutionary Computation: Where Genetic programming fit

Procedural Content Generation (PCG)

- “PCG” = computer-driven creation of game content
- Automates design of levels, items, worlds, and NPCs
- Uses algorithms or AI with little or no human input
- Goal → save more time and reduce manual work
- Balances designer intent with system autonomy

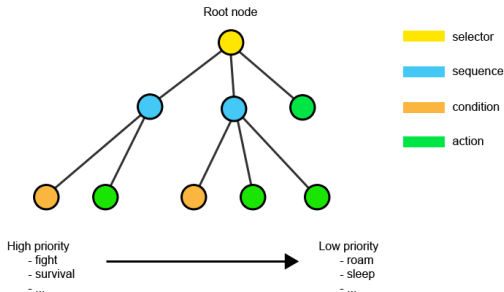
Source: Smith (2014), Understanding Procedural Content Generation

“Algorithmic creation of game content with limited or indirect user input.”

Behavior Trees in Game AI

- Behavior Trees (BTs) organize AI logic into clear, reusable blocks of behavior.
- Each node decides what an NPC should do next, based on checks, priorities, and task results.

Simplified behavior tree



Evolutionary Computation

- **Evolutionary Computation (EC)** methods used:
 - **Genetic Programming** → evolve *Behavior Tree structure* (EvolvingBehavior)
- **EC loop:** Population → Fitness → Selection → Reproduction (Crossover) → Mutation → New generation

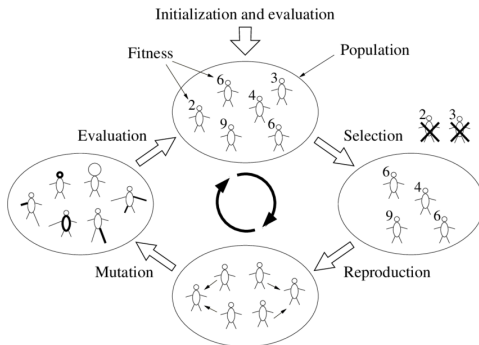


Illustration of the evolutionary computation cycle. [res([n. d.]])

Outline

Background

Evolving Behavior

How the tool works

Experiments

Results

Conclusion

What is EvolvingBehavior

- **EvolvingBehavior:** a co-creative, evolutionary tool inside **Unreal Engine 4** that evolves **Behavior Trees (BTs)** for NPCs. by applying Evolutionary Computation cycle.
- **How it helps:** reducing manual scripting while increasing the variety of behaviors.
- **Designer control:** you provide the seed BT and the fitness goals, testbed and maps pool

Outline

Background

Evolving Behavior

How the tool works

Experiments

Results

Conclusion

How the tool works

1. Designer sets inputs (in UE4):
 - A starting behavior tree (can be partial or even empty).
 - Fitness measures (the scoring rules) and evolution settings.
2. Encode to chromosome:
 - The BT is converted to an editable chromosome
3. Evaluate (game trial):
 - The game runs a short simulation in UE4 survival testbed (zombies vs. humans) across small/medium/large maps with different agent counts.
 - Events update each agent's fitness measures
 - Each individual (BT) gets a fitness score.

How the tool works

4. Select parents (tournament selection):

- To pick one parent, the system randomly samples k individuals (here $k = 4$) and keeps the best ONE of those 4.

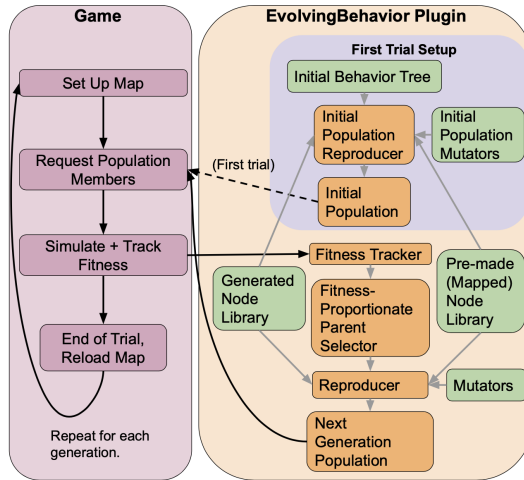
5. Make children:

- Start the child as a copy of the primary parent.
- Crossover: replace a subtree in the child with a subtree from the second parent (swap subtrees)
- Mutations: small random edits (add/replace/delete nodes)

6. New Generation:

- Elitism: small top $n\%$ of the old population unchanged, copied over to keep the best solutions.
- the rest of the old population is replaced by the new children.

How the tool works



An overview of the process and data flow for evolving behavior trees using the EvolvingBehavior tool. Green nodes represent data configured by the designer, pink represent game-specific steps, and orange represent functionality and data provided by the EvolvingBehavior plugin. Grey arrows depict the flow of data within the plugin, whereas black arrows depict the flow related to the game. [Partlan et al. (2022)]

Outline

Background

Evolving Behavior

How the tool works

Experiments

Results

Conclusion

Experiments — Method (1/5)

- **Testbed & goal:** Epic Survival Game (3rd person shooting) by Tom Looman[Looman([n. d.])] → Evolve zombie NPC behavior.
- **Player Stand-ins:** Human character with simple behavior tree → move toward a random point every 0.5s.
- Human stand-ins also prefer paths far from zombies → while also perform brief sprinting with slowing down for recovering stamina.
- **Maps:** Three maps were used → Small (dense obstacles) / Large (sparse obstacles) / Medium (4x small stitched with walls removed)

Experiments — Method (2/5)

- **Initialization for Co-Creative Improvement:** remove patrol/chase from the stock zombie BT → preserve some structure → let evolution fill the movement/chase gaps.
- **Gen 0 (seed):** For each tree, apply up to **10 random edit attempts** drawn from mutation/crossover. **40%** chance of *at least one* mutation and about **40%** chance of *at least one* crossover per tree.
- **Later generations:** Use **12 point-mutation types** (small local edits). Each child has a approximately of 20% chance that at least one of the 12 point-mutation types fires plus lower-rate crossover; keep **12% elitism**.

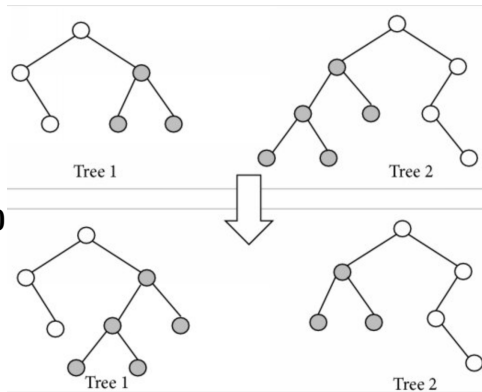
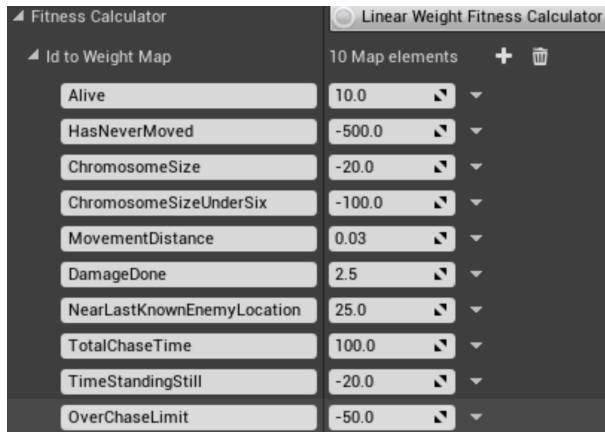


Illustration of subtree swapping (crossover) [Ju et al.(2014)]

Experiments — Method (3/5)

- **Fitness function:** rewards meaningful zombie behavior such as moving/patrolling, initiating and maintaining chases, dealing damage. Penalties for failure movements, or rapidly losing and restarting a chase too many times.



The screenshot shows a software interface titled "Fitness Calculator" with a sub-tab "Linear Weight Fitness Calculator". It features a table with 10 rows, each representing a map element and its assigned weight. The table has three columns: the element name, the weight value, and a dropdown menu for each weight. The weights are: Alive (10.0), HasNeverMoved (-500.0), ChromosomeSize (-20.0), ChromosomeSizeUnderSix (-100.0), MovementDistance (0.03), DamageDone (2.5), NearLastKnownEnemyLocation (25.0), TotalChaseTime (100.0), TimeStandingStill (-20.0), and OverChaseLimit (-50.0). Above the table, there are controls for "10 Map elements", a plus icon to add more, and a trash icon to remove.

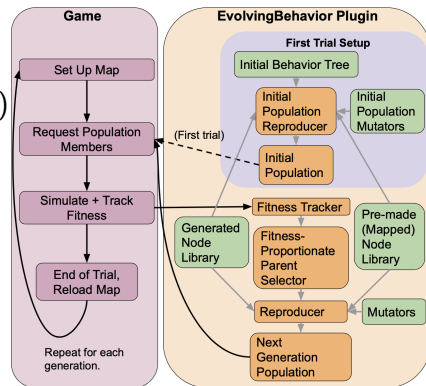
Id to Weight Map	Weight	
Alive	10.0	▼
HasNeverMoved	-500.0	▼
ChromosomeSize	-20.0	▼
ChromosomeSizeUnderSix	-100.0	▼
MovementDistance	0.03	▼
DamageDone	2.5	▼
NearLastKnownEnemyLocation	25.0	▼
TotalChaseTime	100.0	▼
TimeStandingStill	-20.0	▼
OverChaseLimit	-50.0	▼

Experiments — Method (4/5)

- **Behavior Tree nodes:** Composites: Selectors, sequences / Decorators (conditions guard): 8 mapped + 4 generated / Tasks (action node): 13 mapped + 5 generated
- **Experimental conditions:** {Small, Medium, Large} $\times$ {three different counts of human stand-in characters (3, 6, and 12)} resulting a total of 9 conditions
- Zombie counts are different across map: Small=20, Medium=50, Large=40
- Human players respawn 2s at random location.

Experiments — Method (5/5)

- **Run length:** Each evolutionary run proceeded for **1,000 generations**
- **90s simulated** at $15\times$ speed ($\approx$ **6s real time**) to get fitness.
- **Per generation stop:** once new children fill the population, that generation ends and the next begins.
- **Overall stop:** generation budget is reached (1,000 gens) or an optional early-stop.
- **Model selection:** After the run, pick generation with the highest average fitness, then take the highest fitness score individual in that generation as the representative BT for one condition (e.g Small vs 3 human stand-in) repeat across 9 conditions.



An overview of the process and data flow for evolving behavior trees using the EvolvingBehavior tool. [Partlan et al.(2022)]

Outline

Background

Evolving Behavior

How the tool works

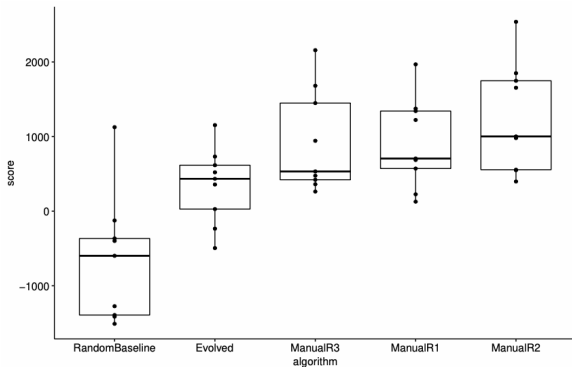
Experiments

Results

Conclusion

Results

- **Evolved BTs** quickly outperform the **random baseline** and stabilize well.
- Across same configs setting, evolved solutions always show **significantly higher** fitness than random.
- **Quality matches hand-authored**
Evolved BTs often **approach hand-authored quality**, especially on patrol → chase patterns.
- **What drives performance:**
Node choice, and fitness function setup with *map density* shape



Box plots of the average fitness results for each algorithm across all problem configurations. [Partlan et al.(2022)]

Outline

Background

Evolving Behavior

How the tool works

Experiments

Results

Conclusion

Conclusion

- Bottom line: **Evolved behavior tree beats random behavior tree** and can **approach human-designed tree** in terms of quality.
- **Takeaway:** Use of **evolutionary design tool** can help reduce the effort and time put in tasks that can be automated, not replacing designer role but keeping them in control and leave space for creativity.
- Support the creation of a wider variety of NPC behaviors

References I

[res([n. d.])] [n. d.].

Models for Evolutionary Algorithms and Their Applications in System Identification and Control Optimization — Scientific Figure on ResearchGate.

https://www.researchgate.net/figure/nitIALIZation-and-the-iterative-cycle-in-evolutionary-algorithms_fig1_2568691

Available from: https://www.researchgate.net/figure/nitIALIZation-and-the-iterative-cycle-in-evolutionary-algorithms_fig1_2568691.

[Grünauer([n. d.])] Philipp Grünauer. [n. d.].

Flexible game AI.

<https://g-phil.com/flexible-game-ai/>

G-Phil: A Game Developers Log.

References II

[Ju et al.(2014)] Ming-Yi Ju, Siao-En Wang, and Jian-Horn Guo. 2014.

Path Planning Using a Hybrid Evolutionary Algorithm Based on Tree Structure Encoding.

TheScientificWorldJournal 2014 (05 2014), 746260.

<https://doi.org/10.1155/2014/746260>

[Looman([n.d.])] Tom Looman. [n.d.].

EpicSurvivalGameSeries: Third person survival game (tutorial) series for Unreal Engine 4.

<https://github.com/tomlooman/EpicSurvivalGameSeries>

GitHub repository.

[Partlan et al.(2022)] Nathan Partlan, Luis Soto, Jim Howe, Sarthak Shrivastava, Magy Seif El-Nasr, and Stacy Marsella. 2022.

EvolvingBehavior: Towards Co-Creative Evolution of Behavior Trees for Game NPCs. In *Proceedings of the 17th International Conference on the Foundations of Digital Games*. Association for Computing Machinery, New York, NY, USA, Article 36, 13 pages.

<https://doi.org/10.1145/3555858.3555896>